

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

ТЗ-8 — WorldTickKernel Unification (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Инструкция по унификации pipeline в единый WorldTickKernel **Тип:** Техническое задание на миграцию (переходный дизайн, не переписывание) **Цель:** свернуть dual pipeline (idle/player) в единый kernel с mode-параметром, устранить divergence в root cause

Принцип (из вашей корректировки): > Не «добавить фазы», а «свернуть в единый kernel». > Phase fix в текущем виде усиливает duplication, закрепляет dual pipeline design, делает divergence хуже при росте системы.

0. Контекст: что патч делает и почему

0.1. Диагноз (уточнённый)

Первоначальный TZ-08 предлагал «добавить 4 пропущенные фазы в player turn». Это было лечением симптомов, не причины. Вы правы:

Подход	Последствия
 Добавить фазы в player_finalize	duplication (одни и те же фазы в 2 местах), закрепление dual pipeline, divergence растёт с NPC complexity
 Свернуть в единый kernel	single simulation contract, idle/player различия исчезают, performance сохраняется

Реальная проблема: не «4 missing phases», а **lack of unified simulation contract across execution modes**.

0.2. Текущая архитектура (что ломается)

execute(campaign_id, scene_state, dm_ctx=None)

- if dm_ctx is None (IDLE):
 - 10 фаз: 0 → 0.5 → 1 → 2 → 3 → 4 → 5 → 6 → 8 → 9 → affective → 10
- if dm_ctx is not None (PLAYER):
 - только 5_player → 6
 - [Rules agent async между вызовами]

└─ execute_player_finalize():
└─ 1 → [directives] → [movement] → 0.5 → 8 → affective → 9 → 10 →
[perception]

ПРОБЛЕМЫ:

1. Phase 2, 3, 4 пропущены в player path
2. Phase 7 (Rules) только в player path (асинх между вызовами)
3. affective pipeline вызывается в 2 местах с разной логикой
4. perception pipeline только в player_finalize
5. nps_contexts пробрасывается вручную (P0_FIX)
6. 2 точки входа (execute + execute_player_finalize) → 2 контекста → divergence

0.3. Целевая архитектура (единый kernel)

WorldTickKernel.execute(state, mode)

- └─ mode = IDLE | PLAYER | SIMULATION
- └─ core_phases = immutable (всегда вызываются одинаково)
- └─ player layer: apply_intervention(state, player_actions)
- └─ rules system: event-driven subscriber (не phase)

ЕДИНЫЙ PIPELINE:

snapshot → 0 → 0.5 → 1 → 2 → 3 → 4 → 5 → 6 → [rules subscriber] → 8 → 9 →
affective → 10 → perception

РАЗЛИЧИЯ ПО MODE:

- IDLE: нет player_actions, нет rules subscriber (или sync rules)
- PLAYER: apply_intervention между 0 и 1, rules subscriber асинх
- SIMULATION: нет player_actions, fast-forward (skip perception/render)

1. Порядок применения (переходный дизайн, не rewrite)

Принцип: не переписывать систему, а постепенно сворачивать dual pipeline в единый kernel. Каждый шаг — independently testable, backward compatible.

STEP 1 → tick_orchestrator.py (A1 — ввести TickMode enum + unified execute)
STEP 2 → tick_orchestrator.py (A2 — _run_core_phases: immutable sequence)
STEP 3 → tick_orchestrator.py (A3 — apply_intervention: player actions как overlay)
STEP 4 → rules_subscriber.py NEW (B1 — Rules как event-driven subscriber)
STEP 5 → game_loop/_init_.py (B2 — убрать ручной вызов Rules agent)
STEP 6 → tick_orchestrator.py (B3 — убрать execute_player_finalize, deprecated)
STEP 7 → tick_orchestrator.py (C1 — affective pipeline в core_phases)
STEP 8 → tick_orchestrator.py (C2 — perception pipeline в core_phases)
STEP 9 → validation tests

Зависимости: - STEP 2 требует STEP 1 (нужен TickMode) - STEP 3 требует STEP 2 (нужен _run_core_phases) - STEP 5 требует STEP 4

(нужен RulesSubscriber) - STEP 6 требует STEP 5 (нужно убрать ручной вызов) - STEP 7-8 требуют STEP 6 (нужно unified execute)

2. PATCH SET A — Unified Kernel (3 шага, ~4 часа)

STEP 1 — backend/app/services/tick_orchestrator.py (A1)

Назначение: Ввести TickMode enum + unified execute signature.

ДОБАВИТЬ в начало файла (после импортов):

```
from enum import Enum
```

```
class TickMode(str, Enum):
```

```
    """Режим выполнения тика. Определяет, какие overlays применяются.

    IDLE      — нет player actions, rules sync (или skip)
    PLAYER    — apply_intervention между фазами, rules async
    SIMULATION — fast-forward, skip perception/render (для СК-style массовой
    симуляции)
    """
    IDLE = "idle"
    PLAYER = "player"
    SIMULATION = "simulation"
```

ЗАМЕНИ signature execute (строка 439):

```
def execute(
    self,
    campaign_id: str,
    scene_state: dict,
    tick_number: int = 0,
    dm_ctx: Optional["DMContextDTO"] = None,
    npc_services: Optional[Any] = None,
    spatial_service: Optional[Any] = None,
) -> Union[TickResultDTO, TickPlayerResultDTO]:
    """Единственная точка входа для тика мира.

    Два режима:
    - idle (dm_ctx=None): полный 10-фазовый цикл без игрока
    - player (dm_ctx=...): фазы 3-6 через legacy pipeline, без 8-10
    """
```

НА:

```
def execute(
    self,
    campaign_id: str,
    scene_state: dict,
```

```

tick_number: int = 0,
mode: TickMode = TickMode.IDLE,
dm_ctx: Optional["DMContextDTO"] = None,
npc_services: Optional[Any] = None,
spatial_service: Optional[Any] = None,
player_actions: Optional[list] = None,
) -> Union[TickResultDTO, TickPlayerResultDTO]:
    """A1-FIX: Единая точка входа для тика мира.

```

Единый pipeline для всех режимов. Различия — только в overlays:

- mode=IDLE: нет player_actions, rules sync
- mode=PLAYER: apply_intervention, rules async
- mode=SIMULATION: fast-forward, skip perception

Backward compat: если dm_ctx передан — mode=PLAYER автоматически.

```

# Backward compatibility: dm_ctx → PLAYER mode
if dm_ctx is not None and mode == TickMode.IDLE:
    mode = TickMode.PLAYER

if scene_state is None:
    return TickResultDTO(status="no_scene")

```

STEP 2 — backend/app/services/tick_orchestrator.py (A2)

Назначение: _run_core_phases — immutable последовательность фаз, вызывается всегда одинаково.

ЗАМЕНИ branching в execute (строки 524-552):

```

try:
    if dm_ctx is not None:
        # Player turn: фазы 5-6 (decision + post-decision)
        self._phase_5_player_decision(ctx)
        self._phase_6_post_decision(ctx)
    else:
        # Idle tick: полный 10-фазовый цикл
        self._snapshot_positions_before(ctx)
        self._phase_0_simulation(ctx)
        self._phase_0_5_idle_services(ctx)
        self._phase_1_input(ctx)
        self._phase_2_event_bus_primary(ctx)
        self._phase_3_memory(ctx)
        self._phase_4_pre_decision(ctx)
        self._phase_5_decision(ctx)
        self._phase_6_post_decision(ctx)
        self._phase_8_drain_secondary(ctx)
        self._phase_9_integration(ctx)
        self._run_affective_pipeline(ctx)

```

```

        self._phase_10_persistence(ctx)
    except Exception as e:
        logger.error(f"[TICK_ORCH] Ошибка в тике {campaign_id}: {e}",
exc_info=True)

```

НА:

```

    try:
        # A2-FIX: единый core pipeline. Режим определяет overlays, не фазы.
        self._run_core_phases(ctx, mode=mode, dm_ctx=dm_ctx,
player_actions=player_actions)
    except Exception as e:
        logger.error(f"[TICK_ORCH] Ошибка в тике {campaign_id}: {e}",
exc_info=True)

```

ДОБАВИТЬ новый метод `_run_core_phases`:

```

def _run_core_phases(
    self,
    ctx: _TickContext,
    mode: TickMode = TickMode.IDLE,
    dm_ctx: Optional["DMContextDTO"] = None,
    player_actions: Optional[list] = None,
) -> None:
    """A2-FIX: Immutable core pipeline. Вызывается всегда одинаково.

    Различия по mode — в overlays (apply_intervention, rules subscriber),
    не в структуре фаз.

    Pipeline:
        snapshot → 0 → 0.5 → 1 → 2 → 3 → 4 → 5 → 6 → [rules] → 8 → 9 → affective →
    10 → perception
    """
    # Snapshot positions (для детекции spatial transitions)
    self._snapshot_positions_before(ctx)

    # Phase 0: Simulation (LifeEngine.tick — idle only; player skip)
    if mode == TickMode.IDLE:
        self._phase_0_simulation(ctx)

    # Phase 0.5: Idle services (decay, social) — всегда
    self._phase_0_5_idle_services(ctx)

    # PLAYER OVERLAY: apply_intervention (directive handling, movement
    intents)
    if mode == TickMode.PLAYER and dm_ctx is not None:
        self._apply_player_intervention(ctx, dm_ctx, player_actions)

    # Phase 1: Input (WillpowerGate) — всегда
    # B IDLE: player_intent=None → no-op
    # B PLAYER: player_intent из dm_ctx

```

```

self._phase_1_input(ctx)

# Phase 2: Event bus primary (spatial events) — всегда
self._phase_2_event_bus_primary(ctx)

# Phase 3: Memory (decay, promotion) — всегда
self._phase_3_memory(ctx)

# Phase 4: Pre-decision (topic extraction) — всегда
self._phase_4_pre_decision(ctx)

# Phase 5: Decision
if mode == TickMode.PLAYER and dm_ctx is not None:
    # Player: делегирует в run_npc_pipeline (legacy)
    self._phase_5_player_decision(ctx)
else:
    # Idle/Simulation: стандартный DecisionHub
    self._phase_5_decision(ctx)

# Phase 6: Post-decision — всегда
self._phase_6_post_decision(ctx)

# Phase 7: Rules — теперь event-driven subscriber (B1), не phase
# (вызывается автоматически через event_bus при combat events)

# Phase 8: Drain secondary (handlers) — всегда
self._phase_8_drain_secondary(ctx)

# Phase 9: Integration (WorldSnapshotBuilder) — всегда
self._phase_9_integration(ctx)

# Affective pipeline — всегда (был в 2 местах, теперь в одном)
self._run_affective_pipeline(ctx)

# Phase 10: Persistence — всегда
self._phase_10_persistence(ctx)

# Perception pipeline — всегда (был только в player_finalize)
if mode != TickMode.SIMULATION:
    self._run_perception_pipeline(ctx)

```

STEP 3 — backend/app/services/tick_orchestrator.py (A3)

Назначение: apply_player_intervention — player actions как overlay, не отдельный pipeline.

ДОБАВИТЬ метод:

```

def _apply_player_intervention(
    self,
    ctx: _TickContext,
    dm_ctx: "DMContextDTO",
    player_actions: Optional[list],
) -> None:
    """A3-FIX: Player actions как overlay на core pipeline.

    Раньше: разбросано по execute_player_finalize (directive handling,
    movement intents, npc_contexts transfer, cognitive overlay).
    Теперь: единая точка — всё player-specific здесь.

    Вызывается между Phase 0.5 и Phase 1 в _run_core_phases.
    """
    # 1. NPC state injection (раньше в execute_player_finalize:706-723)
    _life_engine = self._get_life_engine()
    if _life_engine:
        ctx.npc_states = _life_engine.get_npc_states(ctx.campaign_id)
        if ctx.npc_states:
            _player_entry = next((n for n in ctx.all_npcs_raw if n.get("npc_id") ==
"player"), None)
            ctx.all_npcs_raw = ctx.npc_states
            if _player_entry:
                ctx.all_npcs_raw = [n for n in ctx.all_npcs_raw if n.get("npc_id") !=
"player"]
                ctx.all_npcs_raw.append(_player_entry)

        if not ctx.all_npcs_raw and dm_ctx and dm_ctx.all_npcs_raw:
            ctx.all_npcs_raw = dm_ctx.all_npcs_raw
            logger.info("[CAUSALITY] all_npcs_raw from dm_ctx (LifeEngine cache
empty)")

        # NPIC NORMALIZATION: body_state for all NPCs
        if ctx.all_npcs_raw:
            from app.models.npc_state import BODY_STATE_DISABLED
            for _npc in ctx.all_npcs_raw:
                _bs = _npc.get("body_state")
                if not _bs:
                    _npc["body_state"] = dict(BODY_STATE_DISABLED)

    # 2. Directive handling (раньше в execute_player_finalize:737-810)
    if dm_ctx and hasattr(dm_ctx, 'intent_resolution') and
dm_ctx.intent_resolution:
        _intent_res = dm_ctx.intent_resolution
        _params = _intent_res.original_intent.parameters if
_intent_res.original_intent else None
        _sem_action = getattr(_params, 'semantic_action', None) if _params else
None
        _sem_target = getattr(_params, 'target_reference', None) if _params else
None

```

```

        if _sem_action == "MOVE" and _sem_target:
            _target_ref = _sem_target.lower()
            _is_npc_target = any(
                _target_ref in n.get("name", "").lower() or _target_ref in
n.get("npc_id", "").lower()
                for n in ctx.all_npcs_raw
            ) if ctx.all_npcs_raw else False

        if _is_npc_target:
            try:
                from app.services.social.directive_interpretation_subscriber
import DirectiveInterpretationSubscriber
                import types
                _target_id = getattr(_params, 'target_id', None)
                _directive_payload = {
                    "semantic_action": _sem_action,
                    "target_reference": _target_ref,
                    "target_id": _target_id,
                    "social_pressure": 0.8,
                }
                _mock_event =
types.SimpleNamespace(payload=_directive_payload)
                _directive_deltas =
DirectiveInterpretationSubscriber().handle(_mock_event, ctx.all_npcs_raw)
                if _directive_deltas:
                    ctx.delta_buffer.extend(_directive_deltas)
                    # COGNITIVE OVERLAY: apply deltas immediately (раньше
:786-805)

                    for delta in _directive_deltas:
                        _npc_id = delta.npc_id
                        _npc_state = next((n for n in ctx.all_npcs_raw if
n.get("npc_id") == _npc_id), None)
                        if not _npc_state:
                            continue
                        if hasattr(delta.payload, 'recent_directive_data') and
delta.payload.recent_directive_data:
                            _npc_state.setdefault("perceptual_kernel", {})
["recent_directive"] = delta.payload.recent_directive_data
                        if hasattr(delta.payload, 'stress_delta') and
delta.payload.stress_delta != 0:
                            _npc_state.setdefault("emotion", {})[ "stress" ] =
_npc_state.get("emotion", {}).get("stress", 0.0) + delta.payload.stress_delta
                        if hasattr(delta.payload, 'fear_delta') and
delta.payload.fear_delta != 0:
                            _npc_state.setdefault("social_stats", {})[ "fear_of_player" ] =
_npc_state.get("social_stats", {}).get("fear_of_player", 0.1) +
delta.payload.fear_delta
                        if hasattr(delta.payload, 'shock_impulse') and
getattr(delta.payload, 'shock_impulse', 0.0) > 0.5:

```

```

        _npc_state.setdefault("body_state", {})[ "shock_impulse" ] =
getattr(_npc_state.get("body_state", {}), "shock_impulse", 0.0) +
delta.payload.shock_impulse
        _npc_state.setdefault("body_state", {})[ "consciousness" ] =
max(0.0, 1.0 - delta.payload.shock_impulse)
    except Exception as e:
        logger.error(f"[CAUSALITY_CRASH]
DirectiveInterpretationSubscriber failed: {e}", exc_info=True)

# 3. Movement intents (раньше в execute_player_finalize:816-841)
if ctx.player_result and ctx.player_result.movement_intents:
    from app.services.spatial.movement_engine import MovementEngine
    _spatial_svc = self._resolve_spatial_service(ctx)
    if _spatial_svc:
        me = MovementEngine()
        me.set_spatial_service(_spatial_svc)
        _tick = self.get_current_tick(ctx.campaign_id)
        self._apply_drf_scoring_overlay(ctx.player_result.movement_intents, ctx)
        changes = me.process_intents(
            ctx.player_result.movement_intents, _tick,
            ctx.scene_state.get("npc_positions", {}),
            campaign_id=ctx.campaign_id, scene_state=ctx.scene_state,
        )
        if changes and self._scene_manager:
            self._apply_with_shadow_observation(ctx, changes,
phase_label="REACTIVE_MOVE")

# 4. npc_contexts transfer (раньше в execute_player_finalize:846-876)
if ctx.player_result:
    _pr = ctx.player_result
    if getattr(_pr, 'npc_contexts', None) and ctx.shared_context:
        ctx.shared_context.npc_contexts = _pr.npc_contexts

```

3. PATCH SET B — Rules как Event-Driven Subscriber (3 шага, ~3 часа)

STEP 4 — backend/app/services/events/rules_subscriber.py (NEW FILE)

Назначение: Rules agent как event-driven subscriber, не phase.

Создать файл:

```

"""
path: backend/app/services/events/rules_subscriber.py
Назначение: Rules agent как event-driven subscriber.
Реагирует на combat/action events, применяет D&D 5e checks.
Зависимости: app.services.events.event_bus, app.agents.rules_agent
Основные сущности: RulesSubscriber

```

АРХИТЕКТУРНЫЙ ПРИНЦИП (TZ-08):

*Rules — не phase в pipeline, а subscriber на event_bus.
События combat/attack → RulesSubscriber.handle → D&D checks → deltas.*

Преимущества:

- Работает и в idle, и в player turn (event-driven, не mode-dependent)
- Нет async/sync проблемы (subscriber sync, Rules agent вызывается sync)
- Единая точка для всех rules checks

"""

```
from __future__ import annotations
```

```
import logging
```

```
from typing import Any, Dict, Optional
```

```
logger = logging.getLogger(__name__)
```

```
class RulesSubscriber:
```

```
    """Event-driven Rules agent subscriber.
```

```
    Подписывается на:
```

- PLAYER_ATTACKED, ACTOR_ATTACKS — D&D attack rolls
- COMBAT — damage calculation
- (future) SKILL_CHECK, SAVING_THROW

```
    Не подписывается на:
```

- NPC_MOVED, PROXIMITY_* — не rules events
- DIALOGUE — не rules event

```
    """
```

```
    SUBSCRIBED_EVENTS = {
        "PLAYER_ATTACKED", "ACTOR_ATTACKS", "COMBAT",
        "player_attacks", "actor_attacks",
    }
```

```
    def __init__(self, rules_agent=None):
```

```
        """
```

```
        Args:
```

```
            rules_agent: RulesAgent instance. If None — создаст при первом handle.
```

```
        """
```

```
        self._rules_agent = rules_agent
```

```
    def _ensure_agent(self):
```

```
        """Lazy init RulesAgent. """
```

```
        if self._rules_agent is None:
```

```
            from app.agents.rules_agent import RulesAgent
```

```
            self._rules_agent = RulesAgent()
```

```
        return self._rules_agent
```

```
    def can_handle(self, event_type: str) -> bool:
```

```

"""Может ли subscriber обработать событие."""
return event_type in self.SUBSCRIBED_EVENTS

def handle(
    self,
    event: Any,
    npc_states: list,
    campaign_id: str,
    tick: int,
) -> Optional[Dict[str, Any]]:
    """Обработать событие через Rules agent.

Args:
    event: EventDTO или dict с type, source, payload
    npc_states: список NPC states
    campaign_id: для контекста
    tick: текущий тик

Returns:
    Dict с rules_result (checks, success/failure, damage) или None.
    """

    event_type = getattr(event, 'type', None) or event.get('type', '')
    if not self.can_handle(event_type):
        return None

    try:
        agent = self._ensure_agent()

        # Подготовка context для Rules agent
        _action_type = self._map_event_to_action_type(event_type)
        _rules_context = {
            "classification": [{
                "player": getattr(event, 'source', event.get('source', 'world')),
                "type": _action_type,
            }],
            "tick": tick,
            "campaign_id": campaign_id,
        }

        # Rules agent — sync (не async)
        # Если rules_agent.run sync — вызываем напрямую
        # Если async — нужно asyncio.run (но лучше сделать sync wrapper)
        _actions = [event] # Rules agent ожидает list of actions
        rules_result = agent.run(_actions, _rules_context)

        logger.debug(
            f"[RULES_SUBSCRIBER] event={event_type} → "
            f"action_type={_action_type} → result={rules_result is not None}"
        )

```

```

        return rules_result

    except Exception as e:
        logger.error(f"[RULES_SUBSCRIBER] failed for {event_type}: {e}",
exc_info=True)
        return None

    def _map_event_to_action_type(self, event_type: str) -> str:
        """Маппинг event_type → rules action_type."""
        mapping = {
            "PLAYER_ATTACKED": "attack",
            "player_attacks": "attack",
            "ACTOR_ATTACKS": "attack",
            "actor_attacks": "attack",
            "COMBAT": "attack",
            "combat": "attack",
        }
        return mapping.get(event_type, "player_interacts")

    def attach_to_bus(self, event_bus) -> None:
        """Подписаться на event bus."""
        for event_type in self.SUBSCRIBED_EVENTS:
            try:
                event_bus.subscribe(event_type, self)
                logger.debug(f"[RULES_SUBSCRIBER] subscribed to {event_type}")
            except Exception as e:
                logger.warning(f"[RULES_SUBSCRIBER] failed to subscribe to
{event_type}: {e}")

```

STEP 5 — backend/app/services/game_loop/__init__.py (B2)

Назначение: Убрать ручной вызов Rules agent (Phase 7) — теперь subscriber.

НАЙТИ (строки ~1107-1118):

```

# ФАЗА 7: Rules агент (асинхронный — не зависит от npc_contexts)
_action_type = shared_context.action_type or "player_interacts"
_rules_context = {"classification": [{
    "player": actions[0].player_name,
    "type": self.dm_orchestrator._router.get_rules_action_type(_action_type),
}]}
rules_result = await run_agent_safe(
    "rules", self.rules_agent, (actions, _rules_context), {},
)

```

ЗАМЕНИ на:

```

# B2-FIX: Rules — теперь event-driven subscriber (TZ-08 B1).
# Раньше: ручной вызов Rules agent как Phase 7 (async между execute и
finalize).
# Теперь: RulesSubscriber подписан на event_bus, обрабатывает combat
events автоматически.
#
# Для backward compat: если rules_result нужен в shared_context,
# RulesSubscriber.handle вызывается явно для player action.
rules_result = None
if hasattr(self, '_rules_subscriber') and self._rules_subscriber:
    _action_type = shared_context.action_type or "player_interacts"
    _mock_event = type('Event', (), {
        'type': 'PLAYER_ATTACKED' if 'attack' in _action_type else 'COMBAT',
        'source': actions[0].player_name if actions else 'player',
        'payload': {'action_type': _action_type},
    })()
    rules_result = self._rules_subscriber.handle(
        event=_mock_event,
        npc_states=shared_context.all_npcs_raw or [],
        campaign_id=campaign_id,
        tick=shared_context.current_tick or 0,
    )
else:
    # Fallback: старый путь (если RulesSubscriber не инициализирован)
    logger.warning("[RULES] RulesSubscriber not initialized, using legacy
Phase 7")
    _action_type = shared_context.action_type or "player_interacts"
    _rules_context = {"classification": [{
        "player": actions[0].player_name,
        "type": self.dm_orchestrator._router.get_rules_action_type(_action_type),
    }]}
    rules_result = await run_agent_safe(
        "rules", self.rules_agent, (actions, _rules_context), {},
    )

```

В __init__ GameLoop добавить инициализацию:

```

# B2-FIX: RulesSubscriber init
from app.services.events.rules_subscriber import RulesSubscriber
self._rules_subscriber = RulesSubscriber(rules_agent=rules_agent)
# Подписка на event_bus (если есть)
if hasattr(self, '_event_bus') and self._event_bus:
    self._rules_subscriber.attach_to_bus(self._event_bus)

```

STEP 6 — backend/app/services/tick_orchestrator.py (B3)

Назначение: execute_player_finalize — deprecated, делегирует в unified execute.

ЗАМЕНИ весь метод `execute_player_finalize` (строки 658-915+):

```
def execute_player_finalize(
    self,
    player_result: TickPlayerResultDTO,
    tick_buffer: Any,
    shared_context: Any,
    actions: list,
    campaign_id: str,
    rules_result: Dict[str, Any],
    r3_direct_mode: bool = True,
) -> TickPlayerResultDTO:
    """B3-FIX: DEPRECATED. Делегирует в unified execute.

    Раньше: отдельный pipeline (фазы 8-10 + cognitive overlay + perception).
    Теперь: unified execute с mode=PLAYER обрабатывает всё.

    Backward compat: сохранён для callers, которые ещё не обновлены.
    Внутри — мостирует в execute().
    """

    import logging
    _logger = logging.getLogger(__name__)
    _logger.warning(
        "[DEPRECATED] execute_player_finalize called. "
        "Use execute(mode=TickMode.PLAYER) instead. "
        "This wrapper will be removed in TZ-08 v0.2."
    )

    # Мостируем в unified execute
    _scene_state = shared_context.scene_state if shared_context else {}
    _tick = shared_context.current_tick if shared_context else 0

    # Подготавливаем dm_ctx из shared_context
    _dm_ctx = getattr(shared_context, 'dm_ctx', None)

    result = self.execute(
        campaign_id=campaign_id,
        scene_state=_scene_state,
        tick_number=_tick,
        mode=TickMode.PLAYER,
        dm_ctx=_dm_ctx,
        player_actions=actions,
    )

    # Возвращаем TickPlayerResultDTO (execute уже возвращает правильный тип)
    return result if isinstance(result, TickPlayerResultDTO) else TickPlayerResultDTO()
```

4. PATCH SET C — Consolidate Affective & Perception (2 шага, ~2 часа)

STEP 7 — backend/app/services/tick_orchestrator.py (C1)

Назначение: Affective pipeline — в `_run_core_phases`, убрать дублирование.

Проверить: `_run_affective_pipeline` вызывается в 2 местах (idle:553, player_finalize:887).

В `_run_core_phases` (STEP 2) уже добавлен вызов:

```
# Affective pipeline — всегда (был в 2 местах, теперь в одном)
self._run_affective_pipeline(ctx)
```

Убрать вызов из `execute_player_finalize` (если STEP 6 оставил legacy wrapper — убрать `affective` из него, т.к. `unified execute` уже вызывает).

STEP 8 — backend/app/services/tick_orchestrator.py (C2)

Назначение: Perception pipeline — в `_run_core_phases`, убрать из `execute_player_finalize`.

ДОБАВИТЬ метод:

```
def _run_perception_pipeline(self, ctx: _TickContext) -> None:
    """C2-FIX: Perception pipeline в core_phases.

    Раньше: только в execute_player_finalize (строки 892-905+).
    Теперь: в _run_core_phases для всех режимов (кроме SIMULATION).
    """

    if not ctx.scene_state or not ctx.all_npcs_raw:
        return

    if not hasattr(self, '_manifest_svc'):
        from app.services.perception.behavior_manifestation_service import
BehaviorManifestationService
        from app.services.perception.phenomenology_projection_service import
PhenomenologyProjectionService
        self._manifest_svc = BehaviorManifestationService()
        self._project_svc = PhenomenologyProjectionService()

    try:
        _traces = self._manifest_svc.produce_traces(
            ctx.scene_state, all_npcs_raw=ctx.all_npcs_raw
        )
        _player_perception = self._project_svc.project(
            _traces, ctx.scene_state, tick=ctx.tick_number
        )
```

```

    # Сохранить в shared_context (если есть)
    if ctx.shared_context and hasattr(ctx.shared_context, 'player_perception'):
        ctx.shared_context.player_perception = _player_perception
    except Exception as e:
        logger.warning(f"[PERCEPTION_PIPELINE] failed: {e}")

```

B_run_core_phases (STEP 2) уже добавлен вызов:

```

# Perception pipeline — всегда (был только в player_finalize)
if mode != TickMode.SIMULATION:
    self._run_perception_pipeline(ctx)

```

5. STEP 9 — backend/tests/test_tz8_world_tick_kernel.py (NEW FILE)

"""Integration tests for TZ-08 WorldTickKernel Unification."""

```

import pytest
import inspect
from pathlib import Path

```

class TestPatchA_UnifiedKernel:

"""A1-A3: unified execute with TickMode."""

```

def test_tick_mode_enum_exists(self):
    from app.services.tick_orchestrator import TickMode
    assert TickMode.IDLE == "idle"
    assert TickMode.PLAYER == "player"
    assert TickMode.SIMULATION == "simulation"

def test_execute_has_mode_param(self):
    from app.services.tick_orchestrator import TickOrchestrator
    sig = inspect.signature(TickOrchestrator.execute)
    assert "mode" in sig.parameters, "execute() must have mode param (A1)"

def test_run_core_phases_exists(self):
    from app.services.tick_orchestrator import TickOrchestrator
    assert hasattr(TickOrchestrator, "_run_core_phases"), \
        "_run_core_phases method missing (A2)"

def test_apply_player_intervention_exists(self):
    from app.services.tick_orchestrator import TickOrchestrator
    assert hasattr(TickOrchestrator, "_apply_player_intervention"), \
        "_apply_player_intervention method missing (A3)"

def test_no_dual_branching_in_execute(self):
    """execute() не должен иметь if dm_ctx is not None branching."""
    from app.services.tick_orchestrator import TickOrchestrator
    source = inspect.getsource(TickOrchestrator.execute)
    # Не должно быть прямого branching на dm_ctx

```

```
    assert "if dm_ctx is not None:" not in source or "_run_core_phases" in
source, \
    "execute() still has dual branching (A2 not applied)"
```

```
class TestPatchB_RulesSubscriber:
```

```
    """B1-B3: Rules as event-driven subscriber."""
```

```
    def test_rules_subscriber_exists(self):
        path = Path(__file__).resolve().parents[2] / "app" / "services" / "events" /
"rules_subscriber.py"
        assert path.exists(), "rules_subscriber.py missing (B1)"
```

```
    def test_rules_subscriber_subscribed_events(self):
        from app.services.events.rules_subscriber import RulesSubscriber
        assert "PLAYER_ATTACKED" in RulesSubscriber.SUBSCRIBED_EVENTS
        assert "ACTOR_ATTACKS" in RulesSubscriber.SUBSCRIBED_EVENTS
```

```
    def test_execute_player_finalize_deprecated(self):
        """execute_player_finalize должен быть deprecated wrapper."""
        from app.services.tick_orchestrator import TickOrchestrator
        source = inspect.getsource(TickOrchestrator.execute_player_finalize)
        assert "DEPRECATED" in source or "deprecated" in source, \
            "execute_player_finalize not marked deprecated (B3)"
```

```
class TestPatchC_ConsolidatedPipelines:
```

```
    """C1-C2: affective & perception in core_phases."""
```

```
    def test_run_perception_pipeline_exists(self):
        from app.services.tick_orchestrator import TickOrchestrator
        assert hasattr(TickOrchestrator, "_run_perception_pipeline"), \
            "_run_perception_pipeline method missing (C2)"
```

```
    def test_core_phases_calls_affective(self):
        from app.services.tick_orchestrator import TickOrchestrator
        source = inspect.getsource(TickOrchestrator._run_core_phases)
        assert "_run_affective_pipeline" in source, \
            "affective pipeline not in core_phases (C1)"
```

```
    def test_core_phases_calls_perception(self):
        from app.services.tick_orchestrator import TickOrchestrator
        source = inspect.getsource(TickOrchestrator._run_core_phases)
        assert "_run_perception_pipeline" in source, \
            "perception pipeline not in core_phases (C2)"
```

```
class TestPatchA_PhaseCoverage:
```

```
    """Все фазы вызываются в _run_core_phases."""
```

```

def test_core_phases_calls_all_phases(self):
    from app.services.tick_orchestrator import TickOrchestrator
    source = inspect.getsource(TickOrchestrator._run_core_phases)

    required_phases = [
        "_snapshot_positions_before",
        "_phase_0_simulation",          # IDLE only
        "_phase_0_5_idle_services",
        "_phase_1_input",
        "_phase_2_event_bus_primary",  # ранее пропущена в PLAYER
        "_phase_3_memory",             # ранее пропущена в PLAYER
        "_phase_4_pre_decision",       # ранее пропущена в PLAYER
        "_phase_5_decision",
        "_phase_6_post_decision",
        "_phase_8_drain_secondary",
        "_phase_9_integration",
        "_phase_10_persistence",
    ]

    for phase in required_phases:
        assert phase in source, f"{phase} not in _run_core_phases (TZ-08 not
applied)"

```

6. СВОДКА

Patch	Step	Тип	Время
A1	TickMode enum + execute signature	Foundation	30 мин
A2	_run_core_ph ases (immutable)	Foundation	1.5 часа
A3	_apply_player _intervention (overlay)	Foundation	1 час
B1	rules_subscri ber.py (NEW)	Rules refactor	1 час
B2	game_loop: убрать ручной Rules	Rules refactor	30 мин
B3	execute_playe r_finalize deprecated	Rules refactor	30 мин

Patch	Step	Тип	Время
C1	affective в core_phases	Consolidation	30 мин
C2	perception в core_phases	Consolidation	30 мин
Test	test_tz8_world_tick_kernel.py	—	1 час

Итого: ~7 часов.

7. ЧТО ПАТЧ ГАРАНТИРУЕТ

✓ Единый pipeline (`_run_core_phases`) — immutable, вызывается всегда одинаково
 ✓ TickMode (IDLE/PLAYER/SIMULATION) — различия в overlays, не в фазах
 ✓ Player actions = overlay (`_apply_player_intervention`), не отдельный pipeline
 ✓ Rules = event-driven subscriber, не phase (работает и в idle, и в player)
 ✓ Phase 2, 3, 4 вызываются во всех режимах (раньше пропущены в PLAYER)
 ✓ Affective pipeline — в одном месте (раньше в 2)
 ✓ Perception pipeline — во всех режимах (раньше только в `player_finalize`)
 ✓ `execute_player_finalize deprecated (backward compat wrapper)`
 ✓ Нет dual pipeline divergence

Что НЕ решается (намеренно):

- Полное удаление `execute_player_finalize` (оставлен как deprecated wrapper для backward compat)
 - Async Rules agent (subscriber sync; если Rules agent async — нужен sync wrapper)
 - SIMULATION mode fast-forward (определён, но не реализован — для CK CORE LAYER)
 - Migration всех callers с `dm_ctx` на `mode=TickMode.PLAYER` (постепенно)
-

8. ПРИНЦИПЫ, КОТОРЫЕ ПАТЧ СОБЛЮДАЕТ

1. **Unified simulation contract** — единый pipeline для всех режимов
2. **No duplication** — фазы в одном месте (`_run_core_phases`), не в 2
3. **No dual pipeline** — idle/player различия в overlays, не в структуре
4. **Event-driven rules** — subscriber, не phase
5. **Backward compatibility** — `execute_player_finalize deprecated wrapper`
6. **Performance preserved** — `_run_core_phases` вызывает те же методы, что и раньше

7. **Forward compatible** — TickMode.SIMULATION готов для СК CORE LAYER

9. ФИНАЛЬНАЯ ПРОВЕРКА

1. TZ-08 tests

```
cd backend && python -m pytest tests/test_tz8_world_tick_kernel.py -v
```

2. Existing tests still pass

```
cd backend && python -m pytest tests/ -v --timeout=60
```

3. TickMode enum

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
from app.services.tick_orchestrator import TickMode  
assert TickMode.IDLE == 'idle'  
assert TickMode.PLAYER == 'player'  
assert TickMode.SIMULATION == 'simulation'  
print('TickMode OK')  
"
```

4. _run_core_phases exists

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
from app.services.tick_orchestrator import TickOrchestrator  
assert hasattr(TickOrchestrator, '_run_core_phases')  
assert hasattr(TickOrchestrator, '_apply_player_intervention')  
assert hasattr(TickOrchestrator, '_run_perception_pipeline')  
print('Unified kernel methods OK')  
"
```

5. RulesSubscriber exists

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
from app.services.events.rules_subscriber import RulesSubscriber  
assert 'PLAYER_ATTACKED' in RulesSubscriber.SUBSCRIBED_EVENTS  
print('RulesSubscriber OK')  
"
```

6. execute_player_finalize deprecated

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
import inspect  
from app.services.tick_orchestrator import TickOrchestrator  
source = inspect.getsource(TickOrchestrator.execute_player_finalize)  
assert 'DEPRECATED' in source or 'deprecated' in source  
print('execute_player_finalize deprecated OK')  
"
```

10. ОБНОВЛЁННЫЙ ПОРЯДОК ПРИМЕНЕНИЯ (с TZ-08)

ФАЗА 1 — Фундамент (~23 часа)

- └ TZ-01 (срочные фиксы) ~5 ч
- └ TZ-08 (WorldTickKernel unification) ~7 ч ← НОВЫЙ
- └ TZ-03 (frontend↔backend контракт) ~8 ч

ФАЗА 2 — Deterministic kernel (~6 часов)

- └ Kernel Isolation ~6 ч

ФАЗА 3 — Causal chains (~20 часов)

- └ TZ-02 (каузальная труба воли) ~12 ч
- └ TZ-04 (spatial) ~6 ч
- └ TZ-05 (LLM/DM) ~7 ч

ФАЗА 4 — Cleanup (~10 часов)

- └ TZ-06 (dead code) ~7 ч
- └ TZ-07 (documentation) ~3 ч

ФАЗА 5 — Dynastic simulation (~10 часов)

- └ CK CORE LAYER ~10 ч

ИТОГО: ~69 часов = ~9 рабочих дней

Почему TZ-08 в Фазе 1, перед TZ-02:

TZ-08 — **предусловие** для TZ-02. Нельзя чинить: - BUG-001 (DirectiveSubscriber) — если Phase 2 не вызывается в player turn, directive deltas не доходят - BreakProgressEngine — если Phase 3 (memory) не вызывается, break progress не фиксируется - Spatial events — если Phase 2 не вызывается, NPC_MOVED не детектируется

TZ-08 **разблокирует** остальные causal chain ремонты через unified pipeline.

11. СЛЕДУЮЩИЙ ШАГ (после применения TZ-08)

После подтверждения, что TZ-08 применён и unified pipeline работает:

1. **Kernel Isolation** — поверх unified pipeline (RNG leaks в `_run_core_phases`)
2. **TZ-02** — каузальная труба воли (BUG-001 теперь работает, т.к. Phase 2 вызывается)
3. **TZ-04** — spatial (SpatialQueryService теперь используется во всех режимах)

4. **СК CORE LAYER** — TickMode.SIMULATION уже готов для fast-forward

TZ-08 — **архитектурный фундамент** для всех последующих слоёв. Без него любой ремонт — temporary fix на одном из двух pipelines.

КОНЕЦ ИНСТРУКЦИИ TZ-08